Universidad Peruana Cayetano Heredia
Facultad de Ciencias e Ingeniería
Departamento de Ciencias Exactas

Programación Avanzada 2025

Lab. 11.2. Algoritmos de Ordenación. Parte I.

Junio 06, 2025

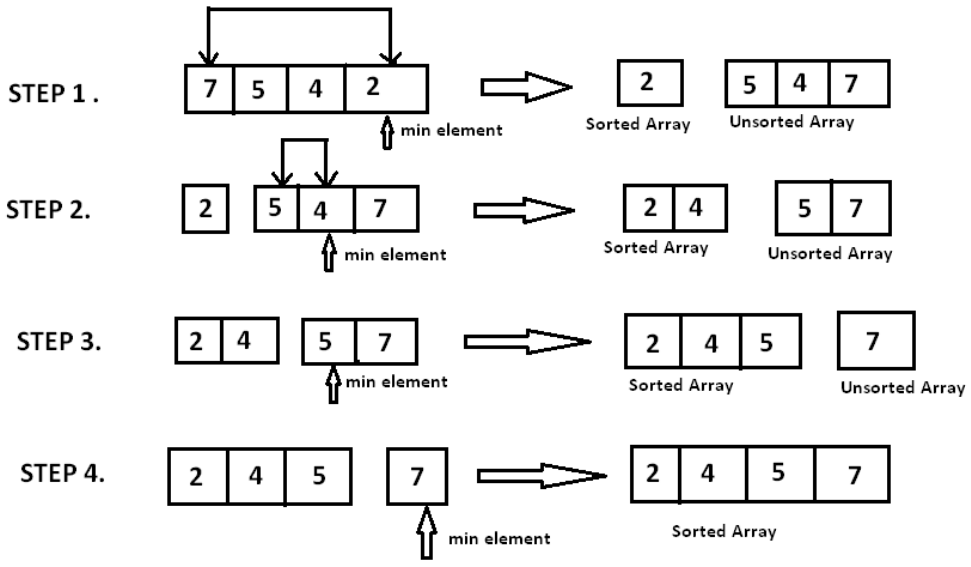
Cree una carpeta (folder) en el disco D, nómbrela con su apellido paterno seguido de su código. Ejemplo: LOPEZ12345

Ordenación es uno de los problemas más estudiados en la Ciencia de la Computación, es así que se ha realizado una vasta investigación en esta área cuyo resultado son los distintos algoritmos que tenemos a disposición en la actualidad.

En este laboratorio veremos tres algoritmos básicos que son fáciles de implementar, pero son ineficientes, luego veremos algoritmos más eficientes. Los ejemplos están basados en secuencias mutables, principalmente listas. Todos los algoritmos ordenarán los valores de la lista de menor a mayor.

Ordenamiento por Selección (Selection Sort)

El algoritmo consiste en encontrar el elemento menor en toda la lista, si el menor no está en la primera posición, se realiza el intercambio entre el menor y el valor que se encuentra en la primera posición. Luego se realiza la misma operación, se busca el menor en la lista a partir de la segunda posición (se acortó el espacio de búsqueda), si el menor no está en la segunda posición, se realiza el intercambio entre el menor y el valor que se encuentra en la segunda posición. La operación continua hasta que la lista quede ordenada de menor a mayor.



He aquí la implementación del algoritmo de ordenación por selección:

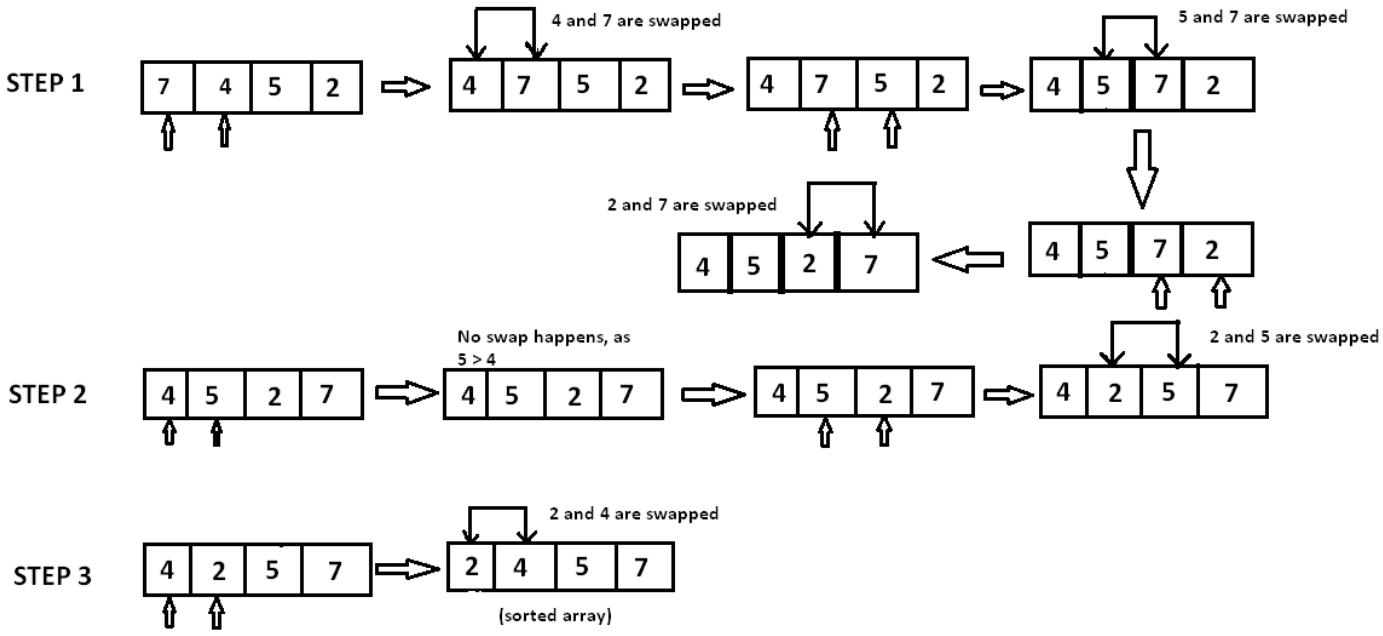
```
# Sorts a sequence in ascending order using the selection sort algorithm.
def selectionSort( theSeq ):
    n = len( theSeq )
    for i in range( n - 1 ):
        # Assume the ith element is the smallest.
        min_idx = i
        # Determine if any other element contains a smaller value.
        for j in range( i + 1, n ):
            if theSeq[j] < theSeq[min_idx] :
                min_idx = j

        # Swap the ith value and smallNdx value only if the smallest value is
        # not already in its proper position. Some implementations omit testing
        # the condition and always swap the two values.
        if min_idx != i :
            tmp = theSeq[i]
            theSeq[i] = theSeq[min_idx]
            theSeq[min_idx] = tmp
```

1. Determinar la complejidad del algoritmo de ordenación por selección, en el peor de los casos, en el mejor de los casos, en el caso intermedio. Indique también las ventajas y desventajas del algoritmo.

Ordenación de Burbuja (Bubble Sort)

El algoritmo consiste en colocar el mayor elemento en la última posición, para lo cual compara los elementos adyacentes de dos en dos, desde el inicio, e intercambia si es necesario. El algoritmo continúa hasta que no sea necesario realizar intercambios.



He aquí la implementación del algoritmo de ordenación de burbuja:

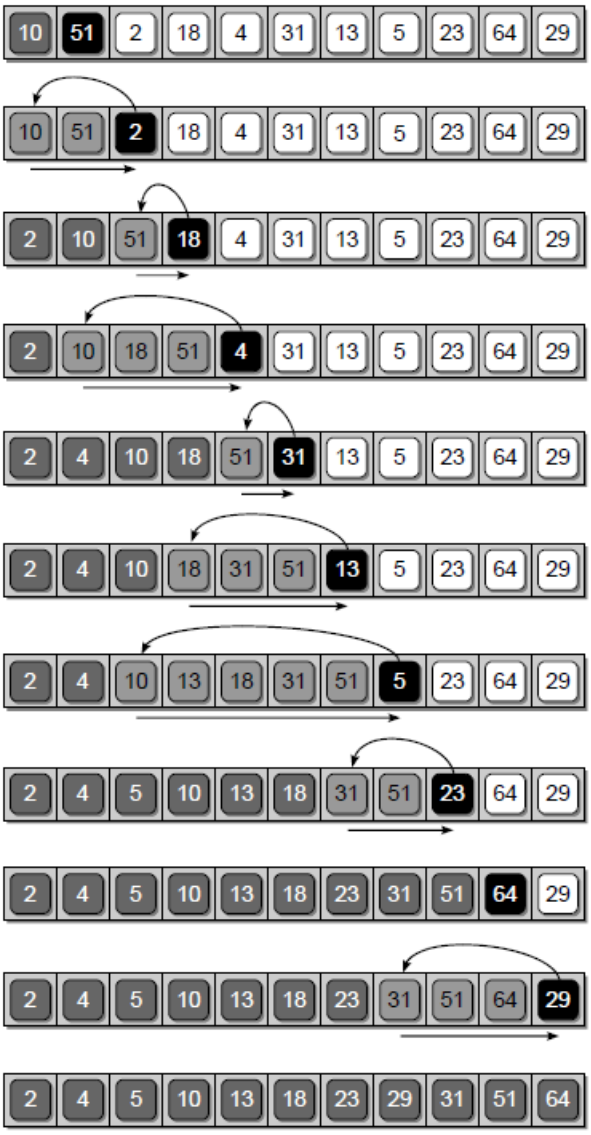
```
# Sorts a sequence in ascending order using the bubble sort algorithm.
def bubbleSort( theSeq ):
    n = len( theSeq )
    # Perform n-1 bubble operations on the sequence
    for i in range( n - 1 ) :
        # Bubble the largest item to the end.
        for j in range( n-i- 1 ) :
            if theSeq[j] > theSeq[j + 1] : # swap the j and j+1 items.
                tmp = theSeq[j]
                theSeq[j] = theSeq[j + 1]
                theSeq[j + 1] = tmp
```

2. ¿Qué pasaría si la lista inicial estuviera ordenada?, en ese caso no habría ningún intercambio de valores. Elabore una versión mejorada del algoritmo detectando si la lista está ordenada.

3. Determinar la complejidad del algoritmo de ordenación de burbuja, en el peor de los casos, en el mejor de los casos, en el caso intermedio. Indique también las ventajas y desventajas del algoritmo.

Ordenación por Inserción (insertion sort)

El algoritmo consiste en recorrer cada elemento de la lista a partir de la segunda posición, para cada elemento verifica si todos los elementos a la izquierda están ordenados y busca la posición que le corresponde y se coloca allí, los demás elementos se desplazan una posición a la derecha. La operación continúa hasta que no haya elementos que analizar.



4. Implementar el algoritmo de ordenación por inserción y determinar su complejidad, en el peor de los casos, en el mejor de los casos, en el caso intermedio. Indique también las ventajas y desventajas del algoritmo.

5. Si un algoritmo $O(n^2)$ (por ejemplo, ordenación de burbuja, ordenación por selección, u ordenación por inserción) toma 3.1 ms para ejecutar sobre un array de 200 elementos, ¿cuánto tomará ejecutar sobre un array similar de:
   a) 400 elementos?
   b) 40,000 elementos?

6. Realice la traza manualmente de la ordenación de la siguiente lista
   A = [44, 88, 55, 99, 66, 33, 22, 88, 77]
   para cada uno de los siguientes algoritmos: Ordenación por selección, por burbuja y por inserción.